Lab Report: Task 1 – Kali Linux & Cryptography

Objective

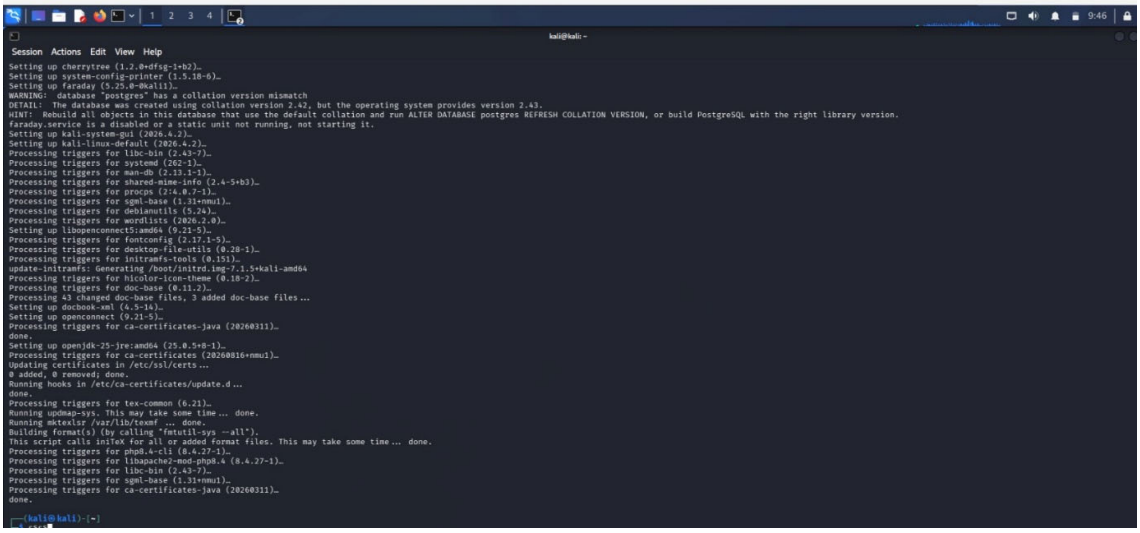
The goal of this lab is to update a Kali Linux system, create a sample confidential text file, and perform both symmetric and asymmetric encryption and decryption using OpenSSL.

1. Updated Kali System

Command used:

sudo apt update && sudo apt upgrade -y

Explanation:
I synchronized the local package database and upgraded the system software packages to ensure the Kali Linux environment was fully up to date.



2. Original File

Command used:

echo -e "--- INTERNAL NETWORK CONFIDENTIAL ---\nSYS_ADMIN_USER=net_admin\nSYS_ADMIN_HASH=\$6\$fakehash\$z8X2...\nINTERNAL_DB_IP=10.0.15.42\nAPI_KEY=sh_live_9a8b7c6d5e4f3g2h1i0j" > confidential.txt

cat confidential.txt

Explanation:
I created a text file named confidential.txt containing realistic simulation data like fake administrative credentials, passwords, and IP addresses to serve as our sensitive data.



3. Symmetrically Encrypted File

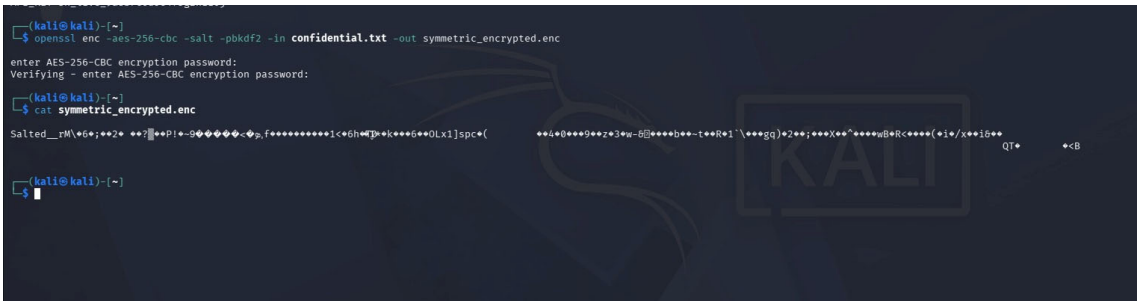
Command used:

openssl enc -aes-256-cbc -salt -pbkdf2 -in confidential.txt -out symmetric_encrypted.enc

cat symmetric_encrypted.enc

Explanation:

I encrypted the confidential file using the AES-256-CBC algorithm. I set up a secure password to scramble the data into unreadable text characters.



4. Symmetrically Decrypted File

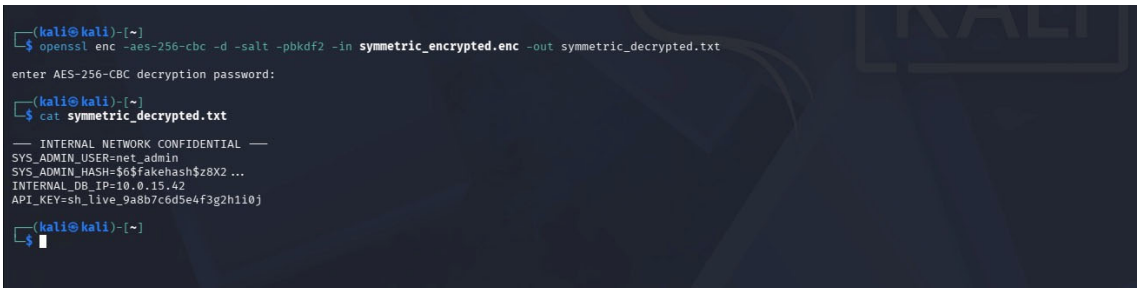
Command used:

openssl enc -aes-256-cbc -d -salt -pbkdf2 -in symmetric_encrypted.enc -out symmetric_decrypted.txt

cat symmetric_decrypted.txt

Explanation:

I used the exact same password to decrypt the scrambled text file back into regular plain text, verifying that the contents perfectly match the original data.



5. Generated Public/Private Key Pair

Command used:

openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048

openssl pkey -in private_key.pem -pubout -out public_key.pem

ls -l *.pem

Explanation:
I switched to asymmetric cryptography and generated a matching 2048-bit RSA key pair: a private key (private_key.pem) to unlock data and a public key (public_key.pem) to lock data.



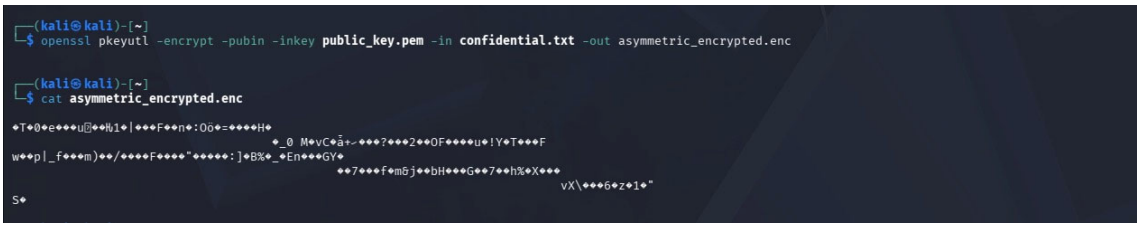
6. Asymmetrically Encrypted File

Command used:

openssl pkeyutl -encrypt -pubin -inkey public_key.pem -in confidential.txt -out asymmetric_encrypted.enc

cat asymmetric_encrypted.enc

Explanation:
I locked the original confidential file using the public key file we generated. The content becomes completely scrambled binary gibberish again.



7. Asymmetrically Decrypted File

Command used:

openssl pkeyutl -decrypt -inkey private_key.pem -in asymmetric_encrypted.enc -out asymmetric_decrypted.txt

cat asymmetric_decrypted.txt

Explanation:
I used my matching secret private key to safely unlock the data, restoring the fake configuration text back to its original readable state.

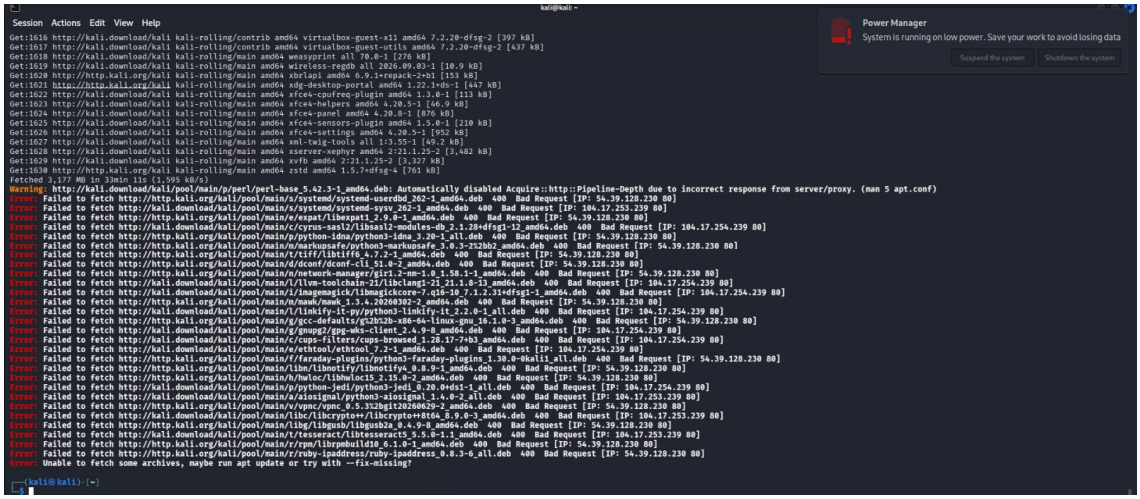
```
┌──(kali㉿kali)-[~]
└─$ openssl pkeyutl -decrypt -inkey private_key.pem -in asymmetric_encrypted.enc -out asymmetric_decrypted.txt

┌──(kali㉿kali)-[~]
└─$ cat asymmetric_decrypted.txt

── INTERNAL NETWORK CONFIDENTIAL ──
SYS_ADMIN_USER=net_admin
SYS_ADMIN_HASH=$6$fakehash$z8X2 ...
INTERNAL_DB_IP=10.0.15.42
API_KEY=sh_live_9a8b7c6d5e4f3g2h110j
```

Challenges Faced

- **Network Pipeline Issue:** During the initial system update phase, the package manager encountered multiple **"400 Bad Request"** errors because the network connection was overwhelmed by simultaneous download streams (HTTP Pipelining).



- **Resolution:** This was resolved by appending the configuration flag -o Acquire::http::Pipeline-Depth=0 to the command line, which forced the system to download files sequentially, bypassing the connection bottleneck.

